

Project Proposal: DodgeBall Boxing Game

Our team will be creating a Mobile Game inspired by the Arcade Boxer genre of Video Games (such as Nintendo's Punch Out). The game consists primarily of two main scenes. First, a Virtual List that allows the user to select a preset for a fight, and second, the actual gameplay part of the application.

The app, like many mobile game applications, is meant to give the user something to do to pass the time or when they are bored. The game is also made with the intent of being easily iterable, allowing further development of the base concept to create a game which can be constantly expanded upon. This ensures that the game will serve its function for an extensive period of time, and can continue to serve it even after a user has "beat" it, since more can always be made.

Course Topics

Mobile UI & React Fundamentals - The app applies React UI and activity principles to build a responsive interface designed for mobile constraints. It handles user input through standard React features and accounts for the app lifecycle to maintain stability during screen transitions.

List Views & Data Mapping - The fight selection system is implemented using a dynamic list structure. Each item represents a different fight configuration, requiring mapping structured data into interactive UI elements. This reflects how list-based interfaces are built and managed in real mobile applications.

Real-Time State & Interaction Design - The gameplay loop continuously updates player health, enemy health, and attack states based on timing and user input. This requires handling frequent state changes and ensuring the UI remains responsive, demonstrating the ability to move beyond static interfaces.

System Architecture & App Flow - The project organizes multiple screens and manages the flow between them (menu to combat). While fully client side, it follows principles similar to client/server separation by structuring data, logic, and UI in a clear and maintainable way.

Team Members	
 Team Member	 Responsibility
Parker Smith	Win & Lose Conditions, Health System
Sam Koenig	User Interface & Design
Riordin Dupree	Enemy States & Attack Tree, List Data

Project Elements

Core Features

1) Fight Menu: Select a Fight From The Fight Menu

Have a list of fight items representing possible fights for the opponent to face. Allow the player to select fights from the list, and to start the fight by pressing a button, which will take the player into Fight Mode.

2) Fight Mode: Attacking The Opponent

Give the player the ability to hit an opponent by tapping either the button at the top or bottom of the enemy image. Attach this button to the enemies health, so that attacks given toward the enemy lower the percentage of remaining health that they have. Create a "win" condition when the opponent's health percentage reaches 0.

3) Fight Mode: Avoiding Enemy Attacks

Give the enemy the ability to attack the player. Attach enemy attacks landing to the players health. Allow the player to avoid the enemies attacks by holding either Guard or Duck. Make the enemy have three types of attacks. An attack can only be avoided by holding Guard, An attack can only be avoided by holding Duck, and an attack which can be avoided by holding either button. The enemy must also always give a tell before attacking to keep the game fair.

Stretch Goals

1) A Score System

Display a high score next to a given fight list element. The score starts out as 0%. The score is defined by the following function (H_E = enemy health & H_P = player health).

$$SCORE\% = (H_P + (100 - H_E)) \div 2$$

2) More Complex Fight Mechanics

This includes adding stun and vulnerable state to enemies, giving the enemy blocking and dodging states, adding more enemy attacks variants beyond the initial 3, and adding a counter to the number of attacks a player has, which is incremented when the player blocks an attack.